



Manual do Programador - Cibersegurança

Guia técnico para manutenção da área de Cibersegurança no projeto central.

Ramo	Cibersegurança
Destinatários	Programadores e responsáveis técnicos
Repositório	https://github.com/MenteMovimento/central-mente-movimento
Site	https://central-mente-movimento.vercel.app
Atualização	22/06/2026

1. Visão geral da Central

A Central MenteMovimento junta os três ramos num único projeto publicado na Vercel e ligado a um único projeto Supabase. A autenticação é centralizada; os dados continuam separados por tabelas e contexto funcional.

- Repositório oficial: <https://github.com/MenteMovimento/central-mente-movimento>
- Site de produção: <https://central-mente-movimento.vercel.app>
- Branch principal: main.
- Build de produção: `npm run build`.
- Diretório publicado pela Vercel: `public`.
- Não publicar ficheiros `.env`, bases SQLite locais, anexos reais, exports com dados pessoais ou chaves privadas.

2. Variáveis e segurança

- `SUPABASE_URL` e `SUPABASE_ANON_KEY` podem ser usadas pelo frontend.
- `SUPABASE_SERVICE_ROLE_KEY` é secreta e deve ficar apenas em Environment Variables da Vercel ou ambiente local protegido.
- `SUPABASE_SECRET_KEY` é usada pelo backend Python de Utentes quando aplicável.
- Depois de mudar variáveis na Vercel, fazer redeploy.
- Se uma chave secreta for exposta, gerar uma nova no Supabase, atualizar Vercel e revogar a antiga.

3. Fluxo de alteração

- 1 Alterar o código localmente.
- 2 Executar `npm run build` na raiz.
- 3 Testar as páginas afetadas.
- 4 Copiar as alterações para a cópia ligada ao GitHub, se estiver a usar a pasta `.publish`.
- 5 Fazer commit com mensagem clara.
- 6 Fazer push para main.
- 7 Verificar o deployment automático na Vercel.

4. Estrutura do ramo Cibersegurança

- Aplicação React/Vite: `portal/modules/dispositivos/src/App.tsx`.
- Estilos: `portal/modules/dispositivos/src/App.css`.
- Build do ramo: `portal/modules/dispositivos/dist`.
- Vite base path: `/area/dispositivos/`.
- Manuais PDF fonte: `portal/modules/dispositivos/public/docs/`.
- Schema Supabase: `portal/modules/dispositivos/supabase/schema.sql`.
- Tabelas principais: `devices`, `profiles`, `device_attachments`.

5. Dados e funcionalidades

- `devices` guarda identificação, hardware, software, estado, diagnóstico, reparação, técnico, datas e observações.
- `device_attachments` guarda metadados dos ficheiros associados a equipamentos.
- O Storage `device-attachments` guarda os ficheiros físicos.
- CSV usa mapeamento de colunas para importar/exportar listas.

- Estatísticas são calculadas a partir dos registos carregados no estado da app.

6. Manutenção React

- Executar `npm --prefix portal/modules/dispositivos run build` depois de alterações.
- Evitar duplicar lógica de importação/exportação em componentes diferentes.
- Ao adicionar campos, atualizar estado inicial, formulário, tabela, CSV, estatísticas e manual.
- Manter botões e ícones alinhados com a barra comum das restantes áreas.

Checklist técnica antes de publicar

- 1 Confirmar que as alterações estão na pasta local correta.
- 2 Executar o gerador de manuais quando houver mudança funcional.
- 3 Executar `npm run build` na raiz do projeto.
- 4 Abrir pelo menos login, dashboard e o ramo alterado.
- 5 Testar navegação entre Sócios, Utentes e Cibersegurança.
- 6 Confirmar que tema escuro, idioma e menu mantêm comportamento global.
- 7 Fazer commit e push para main apenas depois dos testes básicos passarem.

Ficheiros principais deste ramo

- `portal/modules/dispositivos/src/App.tsx` - componente principal, formulários, tabela e estatísticas da área de cibersegurança.
- `portal/modules/dispositivos/src/App.css` - estilos do ramo e adaptação ao tema.
- `portal/modules/dispositivos/public/docs/` - PDFs descarregados no botão Manuais.
- `portal/modules/dispositivos/supabase/schema.sql` - tabelas e políticas do ramo.
- `portal/modules/dispositivos/dist/` - resultado do build React/Vite.

Dados, tabelas e Storage

- Tabela `devices` guarda equipamentos, estado, hardware, diagnóstico e reparação.
- Tabela `device_attachments` guarda metadados dos anexos.
- Bucket `device-attachments` guarda documentos/fotos dos equipamentos.
- Tabela `profiles` pode existir por compatibilidade, mas a gestão global fica em `app_users`.
- CSV deve preservar números de série e identificadores para evitar duplicados.

Regras de compatibilidade

- Não alterar nomes de colunas em produção sem migração SQL e plano de reversão.
- Não remover campos usados por importação, exportação, estatísticas ou histórico.
- Manter a navegação com caminhos absolutos, por exemplo `/area/socios/` e não caminhos relativos.
- Manter textos novos nas tabelas de tradução quando o conteúdo aparece na interface.
- Confirmar que o build da Vercel usa os mesmos ficheiros que foram testados localmente.

Diagnóstico de problemas em produção

- Se só a Vercel falhar, verificar logs do deployment e variáveis de ambiente.
- Se Supabase devolver erro de sessão, verificar `SUPABASE_SERVICE_ROLE_KEY` e permissões.

- Se uma API Python falhar, confirmar importações, caminhos de ficheiros e limites serverless.
- Se um PDF antigo continuar a abrir, limpar cache do browser ou confirmar se o ficheiro foi copiado para public.
- Se houver lentidão, confirmar chamadas repetidas, listas muito grandes e loops de renderização.

Política de dados reais

- Nunca commitar exports reais de sócios, utentes ou cibersegurança.
- Nunca commitar anexos reais ou bases SQLite com dados pessoais.
- Usar dados fictícios em testes e prints de suporte.
- Se uma chave ou ficheiro sensível for publicado por engano, revogar, substituir e registar o incidente.